1. Objetivo del documento

El objetivo de este documento es definir la primera arquitectura técnica estable del ERP para mayorista de turismo. No busca describir todavía todo el sistema final a cinco años, sino dejar resuelto el esqueleto técnico correcto del MVP y de la fase inmediata posterior. Debe servir como puente entre la visión funcional y la implementación real en backend, base de datos y API. La visión general ya dejó claro que el MVP debe cubrir terceros, producto, cotización, reserva, confirmación manual y liquidación básica, mientras que integraciones avanzadas, contabilidad completa y sincronización compleja deben quedarse fuera del núcleo inicial.

---

2. Principios técnicos no negociables

ET-001. Separación real entre cotización y reserva

La base técnica debe reflejar la separación funcional. No se debe "simular" una reserva cambiando simplemente un estado en cotizacion. Debe existir una entidad reserva propia, con sus ítems, sus estados y sus snapshots. Esto ya fue fijado como decisión arquitectónica sana en los documentos funcionales y de reglas.

ET-002. Persistencia de snapshots históricos

Todo valor comercial u operativo que participe en una operación cerrada debe guardarse como fotografía del momento. No se debe confiar en recalcularlo luego desde tablas maestras mutables. Esto aplica especialmente a comisión, precio usado, costo cotizado, costo final y venta confirmada.

ET-003. Baja lógica y auditoría mínima

Las entidades con historial no se eliminan físicamente. Deben tener estado o bandera activo, y toda entidad crítica debe tener al menos created_at, updated_at, y cuando sea posible created_by, updated_by. Esa necesidad ya quedó fijada en las reglas y en el diseño funcional.

ET-004. Regla de negocio crítica en backend y base de datos

El frontend puede ayudar, pero la integridad no puede depender solo de pantalla. Las restricciones estructurales, relaciones, unicidad e integridad deben vivir en base de datos y backend.

---

3. Diagnóstico técnico del SQL actual

La base turismo_db ya tiene una semilla útil y bien orientada para el catálogo y la cotización. Existen las tablas geográficas continente, pais, ciudad; clasificación por tiposervicio; proveedores por proveedor; tabla madre servicio; tablas hijas especializadas como alojamiento, boleto_entrada, tour, traslado, tren y vuelo; además de tablas de tarifas como servicio_precio_mes, hotel_precio_mes, alojamiento_precio_mes y la vista v_alojamiento_precio_mes. También existe el módulo inicial de cotización mediante cotizacion y cotizacion_item.

Eso significa que el SQL actual ya cubre correctamente estas capas del negocio: catálogo básico, especialización por tipo de servicio, parte de la lógica de tarifas base y un primer modelo de cotización con detalle por ítems. Esa base encaja con lo que se definió en los documentos anteriores sobre catálogo de producto, cotización y comportamiento por tipo.

Pero también tiene vacíos importantes para el ERP que queremos construir. No existen todavía tablas propias para cliente_agente, cliente, reserva, reserva_item, confirmacion_comercial, liquidacion, usuarios, roles ni trazabilidad formal por cambio

de estado. Además, cotizacion hoy guarda agente y nombre_pasajero como texto, no como referencias normalizadas a entidades de terceros. Y cotizacion_item tiene FK a cotizacion, pero no tiene todavía FK explícita a servicio, aunque sí tenga índice por id_servicio, lo cual deja una rendija de integridad que conviene cerrar desde ya.
También conviene limpiar duplicidades de diseño pequeñas pero peligrosas. Un ejemplo es catalogo_opcion, que hoy tiene varias UNIQUE superpuestas sobre casi la misma combinación de columnas, algo funcionalmente redundante y poco elegante para mantenimiento.

---

4. Estrategia técnica general para el MVP

La estrategia correcta no es romper todo el SQL actual y empezar otra vez desde cero. Lo sensato es hacer una **refactorización evolutiva**:

1. conservar el bloque geográfico y el bloque de catálogo que ya sirven;
2. ampliar la capa de terceros;
3. fortalecer la capa de cotización;
4. crear la capa nueva de reservas;
5. crear la capa nueva de confirmación comercial;
6. crear la capa nueva de liquidación;
7. añadir auditoría, estados y seguridad mínima.

Esto encaja con la visión modular y con la separación por fases que ya definimos.

---

5. Bloques de entidades del modelo técnico propuesto

El modelo técnico del MVP debe quedar dividido en ocho bloques:

A. Maestros de terceros
B. Maestros geográficos y catálogos
C. Catálogo de producto
D. Tarifas base
E. Cotización
F. Reserva
G. Confirmación comercial y liquidación
H. Seguridad y auditoría

Esa separación respeta la cadena funcional aprobada: clientes y agencias alimentan cotizaciones; servicios alimentan cotizaciones; cotizaciones alimentan reservas; reservas alimentan confirmación comercial; confirmación comercial alimenta liquidación; liquidación alimenta reportes.

---

6. Especificación técnica de maestros de terceros

6.1 Tabla cliente_agente

Esta tabla no existe hoy y debe crearse. Su función es normalizar lo que hoy aparece en forma libre como texto agente en cotizacion. El negocio ya dejó clarísimo que la agencia intermediaria es una entidad central del modelo comercial.
Campos propuestos:

- id bigint unsigned PK
- codigo varchar(20) not null unique
- razon_social varchar(150) not null
- nombre_contacto varchar(120) null
- telefono varchar(40) null

- email varchar(120) null
- direccion varchar(255) null
- nit varchar(60) null
- id_pais int null FK
- id_ciudad int null FK
- comision_base_pct decimal(5,2) null
- activo tinyint(1) not null default 1
- created_at datetime not null
- updated_at datetime not null
- created_by bigint unsigned null
- updated_by bigint unsigned null

Índices:

- uq_cliente_agente_codigo
- idx_cliente_agente_razon_social
- idx_cliente_agente_activo
- idx_cliente_agente_pais

Reglas técnicas:

- codigo debe ser único.
- No se elimina físicamente.
- La comisión base es referencial, no definitiva para histórico.

6.2 Tabla cliente

También debe crearse. Normaliza lo que hoy aparece como nombre_pasajero libre en cotizacion. El diseño funcional exige que el cliente tenga historial de cotizaciones, descartadas y reservas.

Campos propuestos:

- id bigint unsigned PK
- id_cliente_agente bigint unsigned null FK
- nombre_completo varchar(150) not null
- telefono varchar(40) null
- email varchar(120) null
- id_pais int null FK
- id_ciudad int null FK
- observaciones text null
- activo tinyint(1) not null default 1
- created_at datetime not null
- updated_at datetime not null
- created_by bigint unsigned null
- updated_by bigint unsigned null

Índices:

- idx_cliente_nombre
- idx_cliente_agente
- idx_cliente_activo

Reglas técnicas:

- un cliente puede no tener agencia;
- en el MVP un cliente pertenece a cero o una agencia, no a muchas.

6.3 Tabla proveedor_contacto

El proveedor actual está demasiado delgado para el diseño funcional esperado, que pide múltiples correos y contactos. Conviene ampliar mediante tabla hija, no metiendo columnas repetidas en proveedor. El documento operativo ya marcó que el proveedor actual es simplificado y necesita crecer.
Campos propuestos:

- id bigint unsigned PK
- id_proveedor int not null FK
- nombre_contacto varchar(120) null
- email varchar(120) not null
- telefono varchar(40) null
- cargo varchar(80) null
- principal tinyint(1) default 0
- activo tinyint(1) default 1

---

7. Especificación técnica de catálogo y producto

7.1 Qué se conserva

Se conservan tiposervicio, servicio, alojamiento, boleto_entrada, tour, traslado, tren, vuelo, además de geografía y precios base. Esa parte ya está bien alineada con la idea de tabla madre y tablas hijas por tipo.

7.2 Cambios recomendados en servicio

La tabla servicio actual tiene buena base, pero necesita campos de control y algunos ajustes para ERP real. Hoy maneja tipo, proveedor, ciudad, nombre, tiempo, privado, descripción y link.
Campos adicionales recomendados:

- activo tinyint(1) not null default 1
- created_at datetime not null
- updated_at datetime not null
- created_by bigint unsigned null
- updated_by bigint unsigned null

Índices recomendados adicionales:

- idx_servicio_activo_tipo_ciudad
- idx_servicio_proveedor_tipo
- idx_servicio_nombre_busqueda

7.3 Regla técnica para nombres

nombre_wtravel puede seguir siendo único dentro del catálogo activo, pero no recomendaría atarlo como verdad absoluta del universo si más adelante aparecen servicios equivalentes históricos o duplicados lógicos por vigencia. Para el MVP se puede mantener la unicidad, pero conviene prever que en el futuro quizá la unicidad deba pasar a ser compuesta con activo o con una estrategia de versionado. Esa recomendación nace del hecho de que hoy el SQL lo fuerza como único absoluto.

---

8. Especificación técnica de tarifas base

8.1 Qué se conserva

Se conservan servicio_precio_mes, hotel_precio_mes, alojamiento_precio_mes y la vista v_alojamiento_precio_mes, porque ya están bastante bien alineadas con la lógica de precios por periodo que definimos para el catálogo.

8.2 Mejora recomendada

A medio plazo convendrá unificar el modelo de tarifas por vigencia con fechas desde/hasta, pero para el MVP no hace falta romper lo que ya existe. Basta con añadir:

- moneda_codigo char(3) default 'USD'
- activo tinyint(1) default 1
- created_at, updated_at

Y mantener el esquema mensual mientras el producto despega. La visión general ya dejó claro que el MVP solo necesita precios base o mensuales simples, no todavía releases, stop sales o contratación avanzada.

---

9. Especificación técnica de cotización

9.1 Estado actual

cotizacion y cotizacion_item ya existen y forman una base buena para el módulo comercial. Hoy cotizacion guarda agente, pasajero, grupos etarios, fecha de viaje, destino, nombre de cotización, moneda y nota; y cotizacion_item guarda servicio, fecha, orden, opcionalidad, overrides y precio.

9.2 Refactor recomendado para cotizacion

Hoy cotizacion mezcla lógica libre con falta de normalización. Lo correcto para el MVP es ampliar y refactorizar sin perder compatibilidad de migración.

Campos propuestos finales:

- id bigint unsigned PK
- id_cliente bigint unsigned not null FK
- id_cliente_agente bigint unsigned null FK
- nombre_cotizacion varchar(255) not null
- estado enum('ACTIVA','DESCARTADA','CONVERTIDA_RESERVA') not null default 'ACTIVA'
- fecha_viaje_principal date not null
- id_destino_principal int null FK a continente o, mejor, a futuro una entidad de destino
- categoria_viaje varchar(50) null
- moneda_codigo char(3) not null default 'USD'
- presupuesto_estimado decimal(12,2) null
- presupuesto_tipo enum('POR_PERSONA','TOTAL') null
- comision_pct_aplicada decimal(5,2) null
- margen_aplicado_valor decimal(12,2) null
- adultos_65 smallint unsigned default 0
- adultos_19_64 smallint unsigned default 0
- jovenes_12_18 smallint unsigned default 0
- ninos_3_11 smallint unsigned default 0
- infantes_0_2 smallint unsigned default 0
- total_pasajeros smallint unsigned not null
- nota text null
- fecha_descartada datetime null
- fecha_convertida_reserva datetime null
- created_at datetime not null
- updated_at datetime not null
- created_by bigint unsigned null
- updated_by bigint unsigned null

Observación técnica importante: los campos agente y nombre_pasajero deberían migrarse a claves foráneas id_cliente_agente e id_cliente. Eso corrige uno de los principales problemas de diseño actual.
Índices:

- idx_cotizacion_estado_fecha
- idx_cotizacion_cliente
- idx_cotizacion_cliente_agente
- idx_cotizacion_fecha_viaje
- idx_cotizacion_destino_estado

9.3 Refactor recomendado para cotizacion_item

cotizacion_item ya tiene muy buena semilla, pero debe fortalecerse. Actualmente guarda id_cotizacion, id_servicio, fecha, orden, opcionalidad, overrides y precio_usd, y solo tiene FK hacia cotización.
Campos finales recomendados:

- id bigint unsigned PK
- id_cotizacion bigint unsigned not null FK
- id_servicio int not null FK
- fecha_servicio date not null
- orden_dia smallint unsigned not null
- es_opcional tinyint(1) not null default 0
- operador_mostrado varchar(150) null
- link_operador varchar(255) null
- titulo_override varchar(255) null
- clase_override varchar(50) null
- idioma_override varchar(50) null
- nota_linea text null
- precio_sugerido decimal(12,2) null
- precio_venta decimal(12,2) null
- precio_moneda_codigo char(3) not null default 'USD'
- precio_editado_manual tinyint(1) not null default 0
- created_at datetime not null
- updated_at datetime not null

Cambios clave:

- agregar FK real a servicio;
- separar precio_sugerido de precio_venta;
- guardar bandera de edición manual;
- mantener overrides como parte del snapshot comercial.
  Esto está directamente alineado con las reglas RN-087, RN-103 y RN-202.

Índices:

- idx_cot_item_cotizacion_orden
- idx_cot_item_fecha
- idx_cot_item_servicio
- idx_cot_item_opcional

Restricción adicional recomendable:

- unique compuesto por (id_cotizacion,orden_dia,fecha_servicio,id) no hace falta; mejor controlar orden vía backend.

- sí conviene chequear que no haya total_pasajeros inconsistente, pero eso lo resolvería en backend.

---

10. Especificación técnica de reserva

10.1 Necesidad

La reserva no existe hoy como entidad propia y debe crearse. Esto ya había sido identificado como hueco estructural del SQL actual.

10.2 Tabla reserva

Campos propuestos:

- id bigint unsigned PK
- id_cotizacion_origen bigint unsigned not null unique FK
- id_cliente bigint unsigned not null FK
- id_cliente_agente bigint unsigned null FK
- codigo_reserva varchar(50) not null unique
- estado enum('PENDIENTE','PARCIALMENTE_CONFIRMADA','CONFIRMADA','CANCELADA') not null default 'PENDIENTE'
- confirmacion_comercial_estado enum('NO_CONFIRMADA','CONFIRMADA') not null default 'NO_CONFIRMADA'
- fecha_viaje_principal date not null
- id_destino_principal int null FK
- categoria_viaje varchar(50) null
- moneda_codigo char(3) not null default 'USD'
- total_pasajeros smallint unsigned not null
- adultos_65 smallint unsigned default 0
- adultos_19_64 smallint unsigned default 0
- jovenes_12_18 smallint unsigned default 0
- ninos_3_11 smallint unsigned default 0
- infantes_0_2 smallint unsigned default 0
- comision_pct_snapshot decimal(5,2) null
- nota text null
- fecha_conversion datetime not null
- fecha_cancelacion datetime null
- motivo_cancelacion varchar(255) null
- created_at datetime not null
- updated_at datetime not null
- created_by bigint unsigned null
- updated_by bigint unsigned null

Decisiones técnicas:

- id_cotizacion_origen debe ser único en MVP, para cumplir la regla de una reserva principal por cotización;
- la reserva hereda snapshot de la cotización y no debe mutar por cambios posteriores en origen.

Índices:

- uq_reserva_codigo
- uq_reserva_cotizacion_origen
- idx_reserva_estado

- idx_reserva_cliente_agente
- idx_reserva_fecha_viaje

10.3 Tabla reserva_item

Campos propuestos:

- id bigint unsigned PK
- id_reserva bigint unsigned not null FK
- id_cotizacion_item_origen bigint unsigned not null FK
- id_servicio int not null FK
- fecha_servicio date not null
- orden_dia smallint unsigned not null
- es_opcional tinyint(1) not null default 0
- titulo_snapshot varchar(255) null
- idioma_snapshot varchar(50) null
- clase_snapshot varchar(50) null
- nota_linea_snapshot text null
- costo_cotizado decimal(12,2) null
- costo_final decimal(12,2) null
- moneda_codigo char(3) not null default 'USD'
- numero_confirmacion varchar(120) null
- fecha_primer_confirmacion datetime null
- fecha_ultima_confirmacion datetime null
- estado_item enum('PENDIENTE','CONFIRMADO','CANCELADO','NO_APLICA') not null default 'PENDIENTE'
- created_at datetime not null
- updated_at datetime not null

Esto responde exactamente a que cada ítem de reserva guarde costo cotizado, costo final y trazabilidad de confirmación.

Índices:

- idx_res_item_reserva_orden
- idx_res_item_fecha
- idx_res_item_estado
- idx_res_item_num_confirmacion
- idx_res_item_servicio

Regla técnica:

- fecha_primer_confirmacion se setea solo si antes estaba null.
- fecha_ultima_confirmacion se actualiza cada vez que cambie el número.

10.4 ¿Trigger o backend para confirmación?

Para el MVP haría la lógica principal en backend y dejaría un trigger opcional solo para blindar fecha_primer_confirmacion. La razón: el negocio cambiará y es más fácil versionar reglas operativas en backend que en triggers complejos. Aun así, un trigger simple para preservar primera fecha puede ser útil.

---

11. Especificación técnica de confirmación comercial

11.1 Tabla reserva_confirmacion_comercial

Aunque podría resolverse con campos en reserva, recomiendo una tabla propia para preservar snapshots y permitir versionado ligero desde el principio.

Campos propuestos:

- id bigint unsigned PK
- id_reserva bigint unsigned not null FK
- version smallint unsigned not null default 1
- estado enum('ACTIVA','REEMPLAZADA') not null default 'ACTIVA'
- total_venta_confirmada decimal(12,2) not null
- moneda_codigo char(3) not null default 'USD'
- observacion text null
- fecha_confirmacion datetime not null
- confirmed_by bigint unsigned null
- created_at datetime not null

Opcionalmente, una tabla hija reserva_confirmacion_comercial_item:

- id
- id_confirmacion_comercial
- id_reserva_item
- precio_venta_confirmado
- es_opcional_vendido

¿Por qué tabla propia? Porque la regla funcional exige congelar valores finales y distinguir entre preliminar y confirmado. Eso pide snapshot explícito.

---

12. Especificación técnica de liquidación

12.1 Tabla liquidacion_reserva

La liquidación es el espejo financiero del expediente y debe guardar su fotografía de cierre. En el MVP puede haber una por reserva, con posibilidad de recalcular y volver a guardar.

Campos propuestos:

- id bigint unsigned PK
- id_reserva bigint unsigned not null unique FK
- total_venta_confirmada decimal(12,2) not null
- total_costo_real decimal(12,2) not null
- utilidad_bruta_total decimal(12,2) not null
- utilidad_por_persona decimal(12,2) null
- valor_agencia decimal(12,2) null
- diferencia_cotizado_vs_real decimal(12,2) null
- moneda_codigo char(3) not null default 'USD'
- estado_cierre enum('ABIERTA','CERRADA') not null default 'ABIERTA'
- fecha_calculo datetime not null
- fecha_cierre datetime null
- created_at datetime not null
- updated_at datetime not null

Índices:

- uq_liquidacion_reserva
- idx_liquidacion_estado
- idx_liquidacion_fecha

Regla técnica:

- solo se puede crear si existe reserva;
- se recalcula desde datos estructurados, no desde documentos ni totales de pantalla.

13. Especificación técnica de usuarios, roles y auditoría

13.1 Tabla usuario

Campos:

- id bigint unsigned PK
- nombre varchar(120) not null
- email varchar(120) not null unique
- password_hash varchar(255) not null
- activo tinyint(1) not null default 1
- created_at, updated_at

13.2 Tabla rol

Campos:

- id bigint unsigned PK
- codigo varchar(50) unique
- nombre varchar(100) not null

13.3 Tabla usuario_rol

- id_usuario
- id_rol

13.4 Tabla auditoria_evento

Para MVP no hace falta auditar todo el universo, pero sí los eventos críticos:

- creación de cotización,
- descarte de cotización,
- conversión a reserva,
- cambio de estado de reserva,
- carga o cambio de número de confirmación,
- confirmación comercial,
- cierre de liquidación.

Campos:

- id
- entidad varchar(60)
- entidad_id bigint unsigned
- accion varchar(60)
- valor_anterior json null
- valor_nuevo json null
- fecha_evento datetime not null
- id_usuario bigint unsigned null

Esto aterriza lo que el diseño funcional pide sobre usuarios, roles, permisos simples y auditoría mínima.

---

14. Índices prioritarios del MVP

Los índices no deben ponerse como árbol de navidad, sino donde realmente ayudan al uso real del ERP.

Prioritarios:

- búsqueda de clientes por nombre y agencia;
- búsqueda de proveedores por nombre y tipo;
- búsqueda de servicios por tipo, proveedor y ciudad;
- búsqueda de cotizaciones por estado, agencia, cliente y fecha;

- búsqueda de reservas por estado, agencia y fecha;
- búsqueda de reserva_item por número de confirmación y servicio;
- liquidación por estado y fecha.

Ya la base actual trae varios índices buenos en cotizacion, cotizacion_item y precios mensuales; la idea es mantener ese espíritu y completarlo donde faltan entidades nuevas.

---

15. Foreign keys y reglas de integridad recomendadas

Debes blindar estas relaciones desde el inicio:

- cliente.id_cliente_agente -> cliente_agente.id
- proveedor_contacto.id_proveedor -> proveedor.id
- servicio.id_proveedor -> proveedor.id
- servicio.id_tipo -> tiposervicio.id
- servicio.id_ciudad -> ciudad.id
- tablas hijas de servicio -> servicio.id
- cotizacion.id_cliente -> cliente.id
- cotizacion.id_cliente_agente -> cliente_agente.id
- cotizacion_item.id_cotizacion -> cotizacion.id
- cotizacion_item.id_servicio -> servicio.id
- reserva.id_cotizacion_origen -> cotizacion.id
- reserva.id_cliente -> cliente.id
- reserva_item.id_reserva -> reserva.id
- reserva_item.id_cotizacion_item_origen -> cotizacion_item.id
- reserva_item.id_servicio -> servicio.id
- reserva_confirmacion_comercial.id_reserva -> reserva.id
- liquidacion_reserva.id_reserva -> reserva.id

Regla importante: en cascada solo usaría ON DELETE CASCADE en tablas hijas puramente técnicas del mismo agregado cuando la entidad padre nunca deba borrarse en operación real. Para el ERP de negocio prefiero RESTRICT o baja lógica, porque la operación histórica no debería evaporarse por error. Esa preferencia está totalmente alineada con la regla de no borrar datos operativos.

---

16. Estados técnicos recomendados

16.1 Estados de cotización

- ACTIVA
- DESCARTADA
- CONVERTIDA_RESERVA

16.2 Estados de reserva

- PENDIENTE
- PARCIALMENTE_CONFIRMADA
- CONFIRMADA
- CANCELADA

16.3 Estado de confirmación comercial

- NO_CONFIRMADA
- CONFIRMADA

16.4 Estado de liquidación

- ABIERTA

- CERRADA

16.5 Estado común de maestras

- activo booleano

Estos estados salen directamente de las reglas de negocio ya fijadas para cotización, reserva, confirmación y liquidación.

---

17. Campos técnicos transversales recomendados

En casi toda entidad operativa y maestra importante conviene tener:

- id
- activo o estado
- created_at
- updated_at
- created_by
- updated_by

Y en entidades de negocio que cambian de fase:

- fecha_estado
- motivo cuando aplique
- snapshot_* para valores congelados

Esto protege la trazabilidad mínima que ya declaramos obligatoria.

---

18. Decisiones técnicas de backend recomendadas

18.1 Arquitectura sugerida

Para que Codex o Claude no te construyan un monstruo espagueti, el backend debe organizarse por módulos y capas:

- modules/clientes_agentes
- modules/clientes
- modules/proveedores
- modules/servicios
- modules/tarifas
- modules/cotizaciones
- modules/reservas
- modules/liquidaciones
- modules/documentos
- modules/auth

Y dentro de cada módulo:

- controller
- service
- repository
- validator
- mapper
- dto

Esto armoniza muy bien con el diseño modular que ya veníamos usando como criterio general del proyecto.

18.2 Regla de transacciones

Deben usarse transacciones al menos en:

- crear cotización con ítems,
- convertir cotización a reserva,

- registrar confirmación comercial,
- recalcular y cerrar liquidación.

18.3 Regla de snapshots

Al convertir cotización a reserva se copian los datos necesarios.
Al confirmar comercialmente se congelan los importes de venta.
Al liquidar se guarda la fotografía de resultado.
Nunca se debe confiar en recalcular desde maestros mutables.

---

19. Riesgos técnicos que conviene evitar desde ya

El primero es intentar seguir resolviendo todo dentro de cotizacion y cotizacion_item. Eso te dejaría con una pseudo-reserva disfrazada y luego un dolor de cabeza contable. Ya vimos que la propia visión del ERP exige una entidad de reserva separada.
El segundo es no normalizar clientes y agencias. Si sigues guardando agente y nombre_pasajero como texto libre en las operaciones, luego no podrás filtrar, consolidar ni analizar bien por agencia o por cliente.
El tercero es no distinguir precio sugerido, precio de venta, costo cotizado y costo final. Eso rompe la utilidad real y choca directamente con una de las reglas más críticas del proyecto.
El cuarto es intentar meter contabilidad legal completa desde el inicio. La visión general ya dejó claro que el MVP debe concentrarse en operación, confirmación y utilidad básica, no en asientos fiscales ni conciliación bancaria automática.

---

20. Propuesta de implementación técnica por etapas

Etapa técnica 1

Refactor de terceros y cotización:

- crear cliente_agente
- crear cliente
- migrar cotizacion a claves foráneas
- fortalecer cotizacion_item
- añadir auditoría mínima

Etapa técnica 2

Crear reservas:

- reserva
- reserva_item
- lógica de conversión desde cotización
- estados operativos

Etapa técnica 3

Crear confirmación comercial y liquidación:

- reserva_confirmacion_comercial
- liquidacion_reserva
- cálculos de utilidad

Etapa técnica 4

Seguridad y reportes:

- usuarios, roles
- auditoría más sólida
- vistas consolidadas y reportes básicos

Esta secuencia encaja perfectamente con la separación por fases del ERP y con el resumen funcional del MVP.

---

21. Conclusión técnica

La base actual no está mal. De hecho, para catálogo y cotización tiene una semilla bastante buena. El problema no es lo que ya existe, sino lo que todavía no existe: normalización real de terceros, reserva como entidad propia, confirmación comercial congelada, liquidación persistida y auditoría mínima. Eso es exactamente lo que este documento viene a resolver.

La decisión técnica correcta para este ERP es **evolucionar**, no demoler. Mantener el buen bloque de catálogo y cotización, reforzarlo, y construir encima las capas nuevas que convierten el sistema en ERP de verdad. Sin eso, seguiría siendo un cotizador musculoso. Con eso, ya empieza a parecerse a una columna vertebral empresarial.